Pricing Baku Real Estate from Scratch: Decision Trees and Pegasos SVMs on bina.az Listings

Aghazade, Ibrahim Aliyev, Sahib
Mammadli, Mahammad Sakhrayev, Khizri

MLE-AI-201 Machine Learning —
AI Academy, National Artificial Intelligence Center

`github.com/mmahammad57/binaaz-ml-project`

Problem and dataset

- bina.az, a property portal for Azerbaijan: Kaggle collection of 100,775 sale listings, 51 columns
- 91,314 listings after cleaning, 171 features
- **Task A:** regress log(price); skewness $6.00 \rightarrow 0.28$
- **Task B:** premium if price $\geq$ 215,000 AZN, else standard
- Threshold = median *training* price; validation and test reuse it: 50.3 % premium in every split

Data cleaning and leakage

Presenter: Mahammad Mammadli

- Rows removed: 9,412 missing price, area or rooms; 4 duplicates; 45 outliers outside 10k–5M AZN or 10–2,000 m^2
- 30 columns dropped: `unit_price` and `total_price` give the price away; IDs, URLs, times, sellers and free text identify listings
- Floor "k / m" split into floor, storeys and `has_floor`; land area parsed from "sot"
- 5 flag columns encoded 1 = present, 0 = missing
- Seeded, stratified 70/15/15 split: 63,919 / 13,697 / 13,698, one `get_splits()` for every stage

Decision tree from scratch

Presenter: Ibrahim Aghazade

- Gini or entropy (Task B), variance (Task A); split gain

$$\Delta = I(S) - \frac{|S_L|}{|S|} I(S_L) - \frac{|S_R|}{|S|} I(S_R)$$

- Search: every feature, every *observed* value as threshold; largest Δ wins
- Stop: pure node, max. depth, min. rows to split / per leaf, min. gain; tuned: depth 12 (A), 10 (B), leaf 5
- Matches scikit-learn: Task B 464 vs 462 leaves, 99.9 % of test labels agree; Task A predictions correlate 0.9984

Pegasos SVM from scratch

Presenter: Sahib Aliyev

- Objective, labels ± 1:

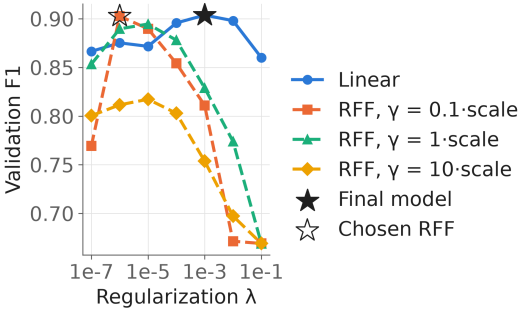
$$J(\mathbf{w}, b) = \frac{\lambda}{2}\|\mathbf{w}\|^2 + \frac{1}{n}\sum_i \max\bigl(0,\, 1 - y_i(\mathbf{w}^\top \mathbf{x}_i + b)\bigr)$$

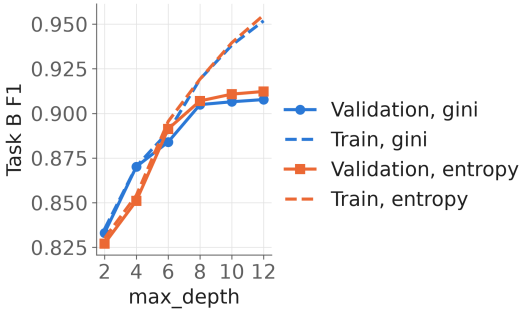
- One random example per step, $\eta_t = 1/(\lambda t)$, then project $\mathbf{w}$ onto $\|\mathbf{w}\| \le 1/\sqrt{\lambda}$:

$$\mathbf{w} \leftarrow (1 - \eta_t \lambda)\,\mathbf{w} + \eta_t\, \mathbf{1}\bigl[y_i(\mathbf{w}^\top \mathbf{x}_i + b) < 1\bigr]\, y_i \mathbf{x}_i$$

- Margin $2/\|\mathbf{w}\|$; support vectors: rows with $y_i(\mathbf{w}^\top \mathbf{x}_i + b) \le 1$
- RFF, $D = 500$: 256 MB instead of a 32.7 GB kernel matrix

Tuning on validation





- Largest λ within 0.005 of the best F1: linear, $\lambda = 10^{-3}$
- RFF: CV F1 0.763 ± 0.095, 320k of 10^6 updates

- Smallest depth, largest leaf within 0.002 of the best
- A: depth 12, leaf 5; B: depth 10, leaf 5

Benchmark against scikit-learn (test set)

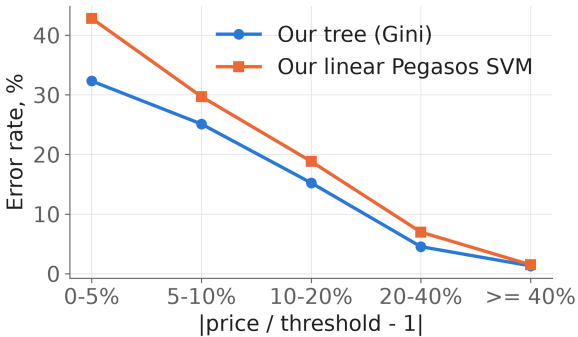
Presenter: Khizri Sakhrayev

- Task A, R^2 on log price: ours 0.870, scikit-learn 0.870

Task B	F1	F1, CV	Fit (s)
Tree Gini (ours)	0.926	0.907 ± 0.006	85.6
Tree Gini (sklearn)	0.926	0.907 ± 0.006	0.28
Pegasos linear (ours)	0.904	0.897 ± 0.003	2.7
SGD hinge (sklearn)	0.905	0.894 ± 0.004	0.29

- Our tree: 305$\times$ slower; fit time grows $\times$15.0 (ours) vs $\times$10.3 (sklearn) from 2k to 16k rows

Where the models fail



Task B error by distance from the threshold

- Within 5 % of the 215,000 AZN threshold: tree 32.3 %, SVM 42.8 % wrong
- That band holds 28 % of tree errors
- Task A median error: 11.0 % cheapest, 18.1 % dearest decile
- Districts with 1–49 vs 1,000+ training rows: 20.2 vs 9.8 %

Limitations and honest notes

- RFF Pegasos is unstable on less data: CV F1 0.763 ± 0.095
- CV on 20,000 of 77,616 rows, because our tree is $305\times$ slower
- Tree sweep on 16,000 rows; Task A's depth is at the grid's top, 12
- One-hot built on all rows: 2 columns all-zero in training
- 0 means "not applicable": floor for 15,338 listings, land area for 76,126

Contributions and next steps

Mahammad Mammadli	data pipeline: cleaning, leakage, split
Ibrahim Aghazade	decision tree, node statistics and the scikit-learn match test, and the tree hyperparameter study (`tree_experiment.py`)
Sahib Aliyev	Pegasos SVM, RFF, sweep and plot tools
Khizri Sakhrayev	metrics, benchmark, SVM tuning, analysis, report

- Faster split search (running sums): tree sweep on all rows
- Bagging or boosting of our trees; location features